

VulGAI: vulnerability detection based on graphs and images

Chunyong Zhang^{a,b}, Yang Xin^{a,b,*}

^a Beijing University of Posts and Telecommunications, Beijing, China

^b National Engineering Lab for DBR, Beijing, China

ARTICLE INFO

Keywords:

Vulnerability detection
Program dependency graph
Node centrality
RGB image
CNN

ABSTRACT

Deep learning models have been widely used in the field of vulnerability detection. Deep learning-based vulnerability detection methods can automatically learn code patterns. Some methods consider processing codes as text sequences to achieve scalable vulnerability detection. They leverage natural language processing models to extract code features. These methods do not consider the code's semantic structure and treat code slices as text. Vulnerability detection methods based on graph structures and graph neural networks are more accurate than text-based methods. However, these methods lack scalability in practice. Both graph generation and graph neural network training are all time-consuming. We propose a **vulnerability** detection method based on **graphs and images** (VulGAI). VulGAI choose the more reasonable node centrality to generate the image. It can preserve program details and distinguish node importance from different perspectives. In addition, we design a more efficient CNN model, which reduces computational overhead and improves detection performance (Time and Accuracy). We implement VulGAI and evaluate six methods (VulDePecker, SySeVR, Devign, VulCNN, mVulPreter, and VulGAI) on 40,657 functions. Experimental results show that VulGAI achieves higher Accuracy, TPR, and F1-Score than the others. In addition, we compare VulGAI and VulCNN on 30270 real-world functions. VulGAI outperforms VulCNN by 1.48 times in the number of TP. VulGAI is about 3.9 times faster than VulCNN in detection time.

1. Introduction

Software vulnerability is an essential security issue in the field of cyberspace security, and code vulnerability is the foundation of software vulnerability. The number of vulnerabilities is increasing year by year, Open Source Security and Risk Analysis (OSSRA) found that 75% of the code base contains vulnerabilities. However, manually checking vulnerabilities becomes very difficult and time-consuming. Therefore, we urgently need an efficient large-scale vulnerability code detection method (Hanif et al., 2021; Ghaffarian and Shahriari, 2017).

With the application and development of deep learning in many fields, researchers have proposed deep learning-based vulnerability detection methods (Li et al., 2018, 2021b; Zou et al., 2019; Chakraborty et al., 2021; Cao et al., 2021; Li et al., 2021a; Fan et al., 2023). Vulnerability detection is mostly regarded as a binary classification problem. Vulnerability detection methods based on deep learning are pattern-based methods. These methods do not need to rely on specific expert knowledge and rules. They use neural networks to automatically learn vulnerability patterns. Li. and Lin. et al. utilized LSTM or BiLSTM to extract input code sequence vectors and detect vulnerabilities (Li et al.,

2018, 2021b; Zou et al., 2019; Li et al., 2021c; Lin et al., 2018, 2019). They belong to the methods based on natural language processing. However, these methods treat code slices as text. They can not reflect the logic and structure of the code.

Research works such as Reveal (Chakraborty et al., 2021), BGNN4VD (Cao et al., 2021), Devign (Zhou et al., 2019), FUNDED (Wang et al., 2020), and Poem (Ye et al., 2020) are devoted to extracting code property graphs (Yamaguchi et al., 2014). Then, they leveraged a gated graph sequence neural network (hereinafter, GGNN (Li et al., 2015)) to extract the vector representation of the graph and perform vulnerability detection. Their scalability is worse than that of NLP-based methods (Li et al., 2018; Zou et al., 2019). In addition, they only focus on whether a function contains a vulnerability. They do not pay attention to the location of the vulnerable code. VulCNN exploits node centrality to preserve program details, which converts code functions into images (Wu et al., 2022). However, katz centrality (Katz, 1953) cannot distinguish nodes. Katz centrality and closeness centrality (Freeman et al., 2002) have duplication. In addition, the CNN model of VulCNN is computationally expensive.

* Corresponding author at: Beijing University of Posts and Telecommunications, Beijing, China.
E-mail address: yangxin@bupt.edu.cn (Y. Xin).

This paper aims to achieve efficient, accurate, and scalable vulnerability detection. We use node centrality to reflect the importance of lines of code. On the one hand, image classification based on CNN models has been developed relatively maturely, and they can guarantee high performance when processing millions of images (Krizhevsky et al., 2017; Rastegari et al., 2016). On the other hand, many techniques for visually interpreting images have emerged. We can leverage these visualization techniques to interpret classification results (Chattopadhyay et al., 2018; Selvaraju et al., 2017). In other words, it might explain the importance of lines of code. However, the performance of CNN models is also very significant in vulnerability detection. We need lower computational overhead and higher detection performance. Therefore, this paper mainly addresses the following two challenges.

- Which node centrality should be chosen to preserve the effective information of the program?
- How to design an efficient CNN model to improve vulnerability detection performance?

To address the above two challenges, we propose a vulnerability detection method based on graphs and images. We first use Joern¹ to process the code into the program dependency graph (Yamaguchi et al., 2014). The program dependency graph contains the data dependencies and control dependencies of codes. In the program dependency graph, we attach each line of code to each node. A graph is a network. The network uses centrality to analyze the importance of nodes (Katz, 1953). However, different centralities have different meanings, and these centralities distinguish nodes differently. Therefore, we analyze multiple centralities (e.g., degree centrality (Freeman et al., 2002), katz centrality (Katz, 1953), closeness centrality (Freeman et al., 2002), second_order centrality (Kermarrec et al., 2011), and PageRank (Le Merrer et al., 2014)). We consider the degree of variation of their values, and the larger the value floats, the greater the distinction between nodes. We selected three centralities of degree centrality, closeness centrality, and second_order centrality to preserve the effective information of the program. Next, we exploit these three centralities to transform the program dependency graph into an RGB image (Wu et al., 2022). Specifically, we use the node vector to multiply the three kinds of centrality to get degree, closeness, and second_order, which correspond to the three channels of R, G, and B, respectively. On the one hand, the three centralities preserve the details of the program from three aspects. On the other hand, three channels work together to generate an image. For a given image, we utilize a CNN model for classification. Specifically, our vulnerability detection model includes jump-structured 2D convolutional neural networks, 1D convolutional neural networks, and fully connected layers. The CNN model with the jump structure can preserve the program details of each layer. This is a new type of vulnerability detection network. To interpret whether a code line is a vulnerable, we use Gradient-weighted Class Activation Mapping++ (hereinafter, Grad-CAM++) (Chattopadhyay et al., 2018) to obtain the corresponding heatmap. This can explain which code line is more likely to be the vulnerable code. Our method can quickly help programmers check for vulnerabilities.

We implemented VulGAI. We evaluated six methods on 40,657 functions, and VulGAI achieved the best performance. In addition, we compare the detection performance, computation performance, and time performance of VulGAI and VulCNN on 30270 real-world function examples. In the correct number of vulnerability detections, VulGAI is 1.48 times more than VulCNN. In terms of the number of parameters, VulGAI is 8.77 times less than VulCNN. In terms of detection time, VulGAI is nearly 3.9 times faster than VulCNN.

In summary, the contributions of this paper are as follows.

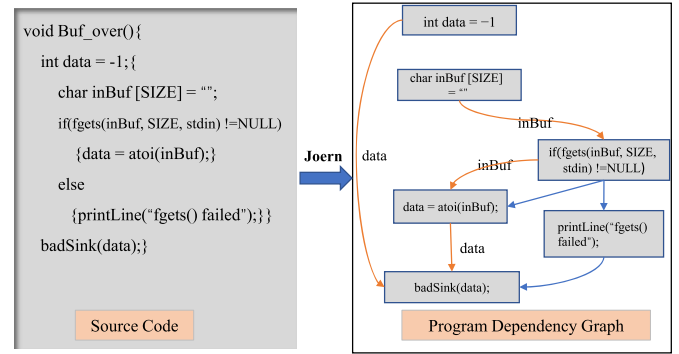


Fig. 1. Program dependency graph of Buffer Overflow vulnerability example. (Yellow lines and blue lines show the data flows and control flows, respectively.)

- We analyzed the shortcomings of VulCNN in the selection of node centrality, and we proposed the more reasonable node centrality to generate images.
- We design a more efficient CNN model, which reduces computational overhead and improves detection performance (Time and Accuracy, etc.).
- In the case study, we comprehensively compare VulGAI and VulCNN with 30,270 function examples. Additionally, we show an example of Buffer Overflow on the RGB image and the interpretability.

Paper organization. The other chapters of this paper are as follows. Section 2 presents the motivation for our improvements. Section 3 introduces the technical route of VulGAI. Section 4 presents the experimental setup and results analysis. Section 5 discusses effectiveness threats, limitations, and future work. Section 6 is related work. Section 7 is the conclusion of this paper.

2. Motivation

Code functions usually consist of multi-line statements. There are different semantic dependencies between them. However, different code statements have different roles in functions. Some lines define the basic variables, and some lines describe the core content of the function (Watson et al., 2022). Therefore, this paper mainly focuses on “vulnerability detection based on the importance of code lines in functions”.

A node in the program dependency graph corresponds to a line of code statement. Its edges have two types of data flow and control flow. The graph structure can well represent the logical relationship and semantic relationship of the code. As shown in Fig. 1. The left side of Fig. 1 is a Buffer Overflow vulnerability code example, and the right side is the program dependency graph of the code. Where the orange solid line represents the data flow, and the blue solid line represents the control flow.

VulCNN has demonstrated that node centrality is helpful for vulnerability detection (Wu et al., 2022). VulCNN exploits node centrality to consider the importance of nodes in the program dependency graph, that is, the contribution of code lines to program details. We will introduce five kinds of centrality: degree centrality, closeness centrality, second_order centrality, katz centrality, and PageRank. Degree centrality refers to the number of connections or degrees between a node and other nodes (Freeman et al., 2002).

Degree centrality is the most direct measure to characterize the centrality of nodes. The greater the degree centrality, the more important the node is.

$$d_i = \frac{D_i}{N - 1} \quad (1)$$

¹ <https://joern.io/>.

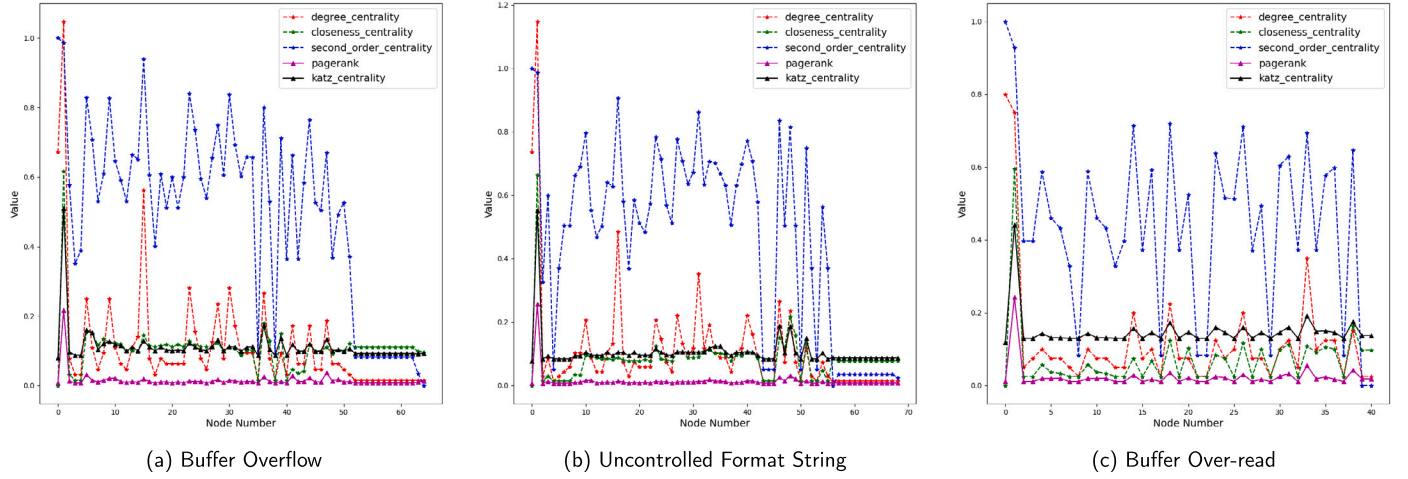


Fig. 2. Five centralities (degree centrality, closeness centrality, second_order centrality, katz centrality, and PageRank) of the three vulnerability examples.

Where N represents the number of nodes, and D_i represents the degree of node i .

Closeness centrality (Freeman et al., 2002) considers the average length of the shortest path from each node to other nodes. That is, for a node, the closer it is to other nodes, the higher its centrality.

$$c_i = \frac{N-1}{\sum_{j=1}^N l_{ij}} \quad (2)$$

Where l_{ij} represents the distance between nodes i and j .

Second_order centrality (Kermarrec et al., 2011) describes the standard deviation of the return times of a node's random walk. Each node records the time elapsed between visits of that random walk and computes the standard deviation (or second order moment) of such return times. The key point is that central nodes see regularly the random walk compared to other topology nodes. Standard deviation can be used to accurately identify critical nodes as well as to globally characterize graphs topology. Lower values of second-order centrality indicate higher centrality. In this paper, the second_order centrality is normalized, and the larger the value after normalization, the higher the centrality.

$$s_i = 1 - \sqrt{\frac{1}{N-1} \sum_{k=1}^N \theta_i(k)^2 - \left[\frac{1}{N-1} \sum_{k=1}^N \theta_i(k) \right]^2} \quad (3)$$

Where $\theta_i(k)$ indicates the time when node i returns for the k^{th} time.

Katz centrality considers the combined effects of path length and link strength between a node and its neighbors. It iteratively considers the paths between a node and its neighbors. The calculation formula of Katz centrality is as follows:

$$x_i = \alpha * \sum A * x_j * \beta \quad (4)$$

Where α is the attenuation factor, A is the adjacency matrix, x_j is the neighbor node of node i , and β is the initial center value of the node.

PageRank calculates a node's importance score by considering the contributions of the node's neighbors and the strength of the links. The calculation process of the PageRank score is iterative. In each iteration, each node updates the importance score of the node according to the importance scores of its neighbor nodes and finally generates a node ranking to reflect the importance of the node. The PageRank value of node i is calculated as follows.

$$PR_i = \frac{1-d}{N} + d * \sum_j \frac{PR_j}{L_j} \quad (5)$$

Where d is the damping factor, N is the number of nodes, PR_j is the pagerank score of node j , j is the neighbor node of node i , and L_j represents the out-degree of node j .

VulCNN converts degree centrality, closeness centrality, and katz centrality into three channels (RGB) of an image. It utilizes TextCNN for classification. Inspired by VulCNN, we analyzed the five centralities (e.g., degree centrality (Freeman et al., 2002), katz centrality (Katz, 1953), closeness centrality (Freeman et al., 2002), second_order centrality (Kermarrec et al., 2011), and PageRank (Le Merrer et al., 2014)) of the three vulnerability examples. They are Buffer Overflow vulnerability, Uncontrolled Format String vulnerability, and Buffer Over-read vulnerability. As shown in Fig. 2. Fig. 2a is an example of a Buffer Overflow vulnerability, Fig. 2b is a vulnerability example of the Uncontrolled Format String, and Fig. 2c is an example of Buffer Over-read. From these three examples, we draw the following conclusions.

- From Fig. 2a and Fig. 2b, we can find that the black line (katz centrality) and the green line (closeness centrality) almost coincide, and their values have little difference. Therefore, the two centralities are repetitive.
- From the three examples, we found that the difference between katz centrality and the PageRank of many nodes is very small (black lines and purple lines). Therefore, katz centrality is not suitable for distinguishing nodes.
- The difference between closeness centrality in Fig. 2a and Fig. 2b is very small. However, closeness centrality in Fig. 2c can still distinguish different nodes. Therefore, we still retain the closeness centrality in this paper.
- The degree centrality and second_order centrality of each node are very different, which can completely distinguish the importance of nodes. This seems reasonable since there are different dependencies between lines of code.

In this paper, the second_order centrality is normalized, and the larger the value after normalization, the higher the centrality. We analyze the shortcomings of katz centrality. Therefore, we utilize closeness centrality, degree centrality, and second_order centrality to analyze the contribution of nodes in the whole program dependency graph. Finally, we redesigned the CNN model to improve computational performance and time performance.

3. VulGAI methodology

In this section, we briefly introduce VulGAI, an improved and efficient static code vulnerability detection method. VulGAI includes four stages: Graph Generation, Node Embedding, Image Generation, and

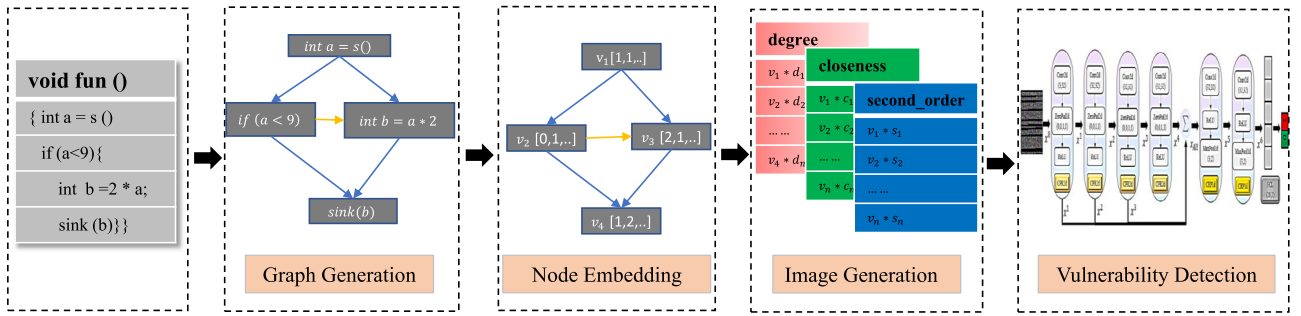


Fig. 3. VulGAI overview.

Vulnerability Detection. As shown in Fig. 3. The graph generation phase extracts the source code as a program dependency graph. In the node embedding stage, we utilize Sent2vec (Pagliardini et al., 2018) to achieve embedding. The image generation stage is to use degree centrality, closeness centrality, and second_order centrality to obtain an RGB image. The vulnerability detection stage is a binary classification. We train the jump-structured CNN model for classification.

3.1. Problem formalization

A function can also perform a specific task. Therefore, this paper detects code vulnerabilities at the functional level, which is a more fine-grained detection method than files. This paper treats the problem of vulnerability detection as a binary classification problem. That is, given a code function, we detect whether it contains the vulnerable code. In this paper, node centrality is used to convert the code into an image, the CNN model is used to classify the image. We presume a set of image training data to be represented as (I_i, y_i) , where I_i represents the source code, and y_i represents the source code label, that is, vulnerability or neutral (0 or 1). The goal of this paper is to design a vulnerability detection model F to detect vulnerabilities. F needs to reduce the value of the loss function $\mathcal{L}$ through continuous iterative learning. In form,

$$\min \sum_{i=1}^n \mathcal{L}(F(I_i, y_i)) \quad (6)$$

3.2. Graph generation

VulGAI is a function-level vulnerability code detection method, and we expect a code to correspond to a graph. Therefore, the goal of this phase is to generate a program dependency graph. However, there are annotations, custom variable names, and function names in the source code. Therefore, we perform the following steps before parsing the code (Zou et al., 2022).

Step 1: Remove annotations to prevent annotation noise.

Step 2: Change the personalized variable name to VAR1, VAR2, ..., such as, $(a = \text{atio}(b) \rightarrow \text{VAR1} = \text{atio}(\text{VAR2}))$.

Step 3: Change the personalized function name to FUN1, FUN2, ..., such as, $(\text{VAR1} = \text{FUN1}(\text{VAR2}) \rightarrow \text{VAR1} = \text{FUN1}(\text{VAR2}))$.

After the above three steps, we have realized the cleaning and abstraction of the source code. Next, we use Joern to parse C/C++ functions. Joern can parse individual functions and analyze the semantic relationship of functions. It usually identifies data dependencies and control dependencies. Finally, we obtain a program dependency graph for a single function. The nodes of the program dependency graph correspond to the statements in the code, and its edges contain data flow and control flow. As shown in the program dependency graph in Fig. 3.

3.3. Node embedding

The goal of this stage is to obtain the vector embedding of the code statement in the node, such as Node Embedding in Fig. 3. In gen-

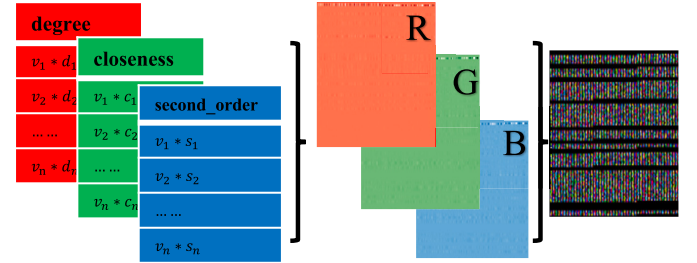


Fig. 4. An image of a Buffer Overflow vulnerability.

eral, there are corresponding node embedding methods for graph nodes (e.g., Node2vec (Grover and Leskovec, 2016), DeepWalk (Perozzi et al., 2014), and Stru2vec (Ribeiro et al., 2017) et al.). However, we found that the performance of these methods is lower than Sent2vec. Therefore, we use Sent2vec to implement unsupervised embedding to obtain a fixed-length vector v_i of nodes. Node vectors have a fixed length of 128.

3.4. Image generation

After the node embedding phase, we get a program dependency graph with node vectors. The goal of this stage is to use node centrality and node vector values to get three channels (RGB) of an image. We consider the centrality and importance of nodes in the whole graph structure. Centrality was first used in social networks (Freeman et al., 2002). However, a network is a complex graph. A program dependency graph is also a graph structure. The concept of centrality can also be used to describe the degree of contribution of code statements. In recent years, many centralities have been used to describe the importance of nodes in a network. Based on the motivation, we use degree centrality, closeness centrality, and second_order centrality to describe the importance of nodes (the contribution of code sentences). Specifically, we can get the degree centrality, closeness centrality, and second_order centrality of the nodes in the program dependency graph through Eq. (1), (2), (3), we multiply it with the node vector v_i to get degree, closeness, and second_order.

$$\text{degree} = v_i * d_i \quad (7)$$

$$\text{closeness} = v_i * c_i \quad (8)$$

$$\text{second_order} = v_i * s_i \quad (9)$$

They correspond to the three channels of the image respectively. That is, degree corresponds to R, closeness corresponds to G, and second_order corresponds to B. Three channels will generate an image. Fig. 4 is the image generation process of the Buffer Overflow vulnerability. We calculated the importance of nodes in the program dependency graph from three perspectives. Therefore, we can analyze the contribution of the line of code in the whole function.

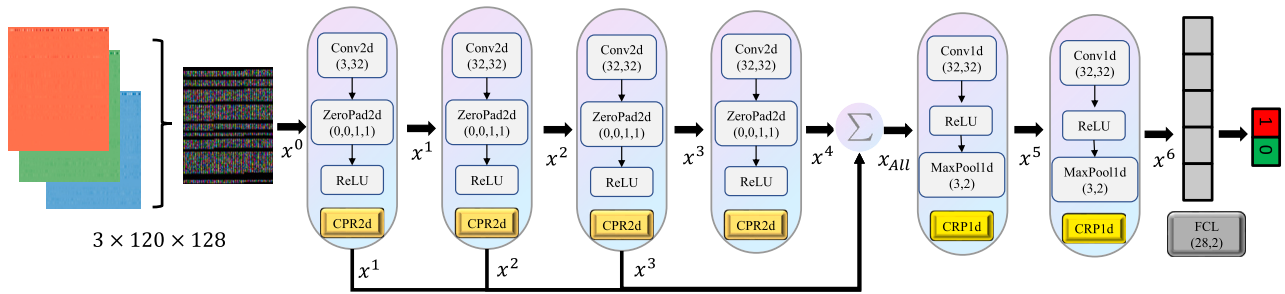


Fig. 5. Vulnerability detection.

3.5. Vulnerability detection

After the previous stages, we obtain an RGB image, which is an image representation of a code. In the field of image recognition, CNN improves model performance with its unique fine-grained feature extraction. The purpose of converting the graph structure into an image is to take advantage of the efficiency of CNN in the image field. To reduce model parameters and improve time performance, we made several simple model attempts. In the end, we use a two-dimensional convolutional neural network with a jump structure to retain the feature information of the image in the upper layer of the network. The 2D convolutional neural network can pass through multiple convolutional layers and activation functions, and the model can extract more abstract and advanced features. In addition, we utilize 1D convolutional neural networks and pooling layers to extract different local features of images. The CNN model and network parameters are shown in Fig. 5. Specifically, in the 2D convolutional neural network, we include four CPR2d(Conv2d + ZeroPad2d + ReLU) modules, each CPR2d contains a Conv2d, a ZeroPad2d and an activation function ReLU. We exploit the jump structure to preserve the image information from the previous layer. Among them, the first Conv2d input and output size is (3,32), and the kernel_size is (3,128). However, the input and output sizes of the other three Conv2d are (32,32), and the kernel_size is (3,1). In the 1D convolutional neural network, we include two CRP1d(Conv1d + ReLU + MaxPool1d) modules, each CRP1d contains a Conv1d, a ReLU, and a MaxPool1d. The input and output size of Conv1d is (32,32), the kernel_size of MaxPool1d is 3, and the stride is 2. Finally, we add a fully connected layer for classification.

CNN needs to take images of the same size as the input. However, the number of lines in the source code is not a fixed value. We tested the dataset (experimental section), and we recorded the number of lines of each function. 97.6% of the function codes were within 120 lines, and 99% of the function codes were within 200 lines.

In addition, we get the vulnerability detection performance of 50, 80, 100, 120, 150, and 200 lines in Experiment 2. We find that when the line threshold is 120, the vulnerability detection performance reaches a steady state. Therefore, this paper chooses 120 lines as the threshold to generate the image. For codes longer than 120 lines, we will remove the redundant part of the vector. For codes less than 120 lines, we add 0 to the end of the vector. Therefore, the size of our generated image is $3 \times 120 \times 128$. Where 3 represents 3 channels (degree, closeness, and second_order), 128 corresponds to the vector length of the node, and 120 corresponds to the line threshold.

4. Experiments

In this section, we will introduce the research questions, experimental settings, experimental methods, and experimental results of this paper.

4.1. Research questions

This paper proposes the following three research questions for the vulnerability detection task.

RQ1: How is the vulnerability detection performance of VulGAI?

RQ2: What are the effects of line threshold, CPR2d layer number, and node embedding method on VulGAI?

RQ3: What is the vulnerability detection performance of VulGAI in the real world?

4.2. Experimental setup

4.2.1. Experimental environment

This paper experiments on a server with Intel(R) Xeon(R) CPU E5-2430 0 @ 2.20 GHz and 256 GB of running memory. For the development environment, we use Python 3.7. For program dependency graph generation, we use version 0.6.1 of Joern. For the deep learning library, we use version 1.13 of the torch library.² For centrality calculations, we use version 2.6.3 of networkx.³ In the vulnerability detection model stage, VulGAI uses Cross Entropy Loss and Adam optimizer. The batch size is 32, the epoch number is 100, and the learning rate is 0.001.

4.2.2. Experimental data

The quality of the dataset determines whether a good model can be trained. We need to choose a dataset with a large amount of data and balanced types for experiments. SARD⁴ is a vulnerability collection project maintained by the National Institute of Standards and Technology. National Vulnerability Database (NVD)⁵ is a vulnerability project maintained by the United States, which contains a large number of vulnerability codes and neutral codes. This paper is a function-level C/C++ vulnerability detection method. Therefore, we use the dataset of SARD_NVD. The dataset contains 13687 vulnerable functions and 26970 neutral functions. The main types of vulnerabilities contained in SARD_NVD are OS Command Injection (CWE-78), Stack-Based Buffer Overflow (CWE-121), Buffer Underwrite (CWE-124), Use of Externally-Controlled Format String (CWE-134), Free Memory Not on Heap (CWE-590), Buffer Under-read (CWE-127), Unexpected Sign Extension (CWE-194), Heap-Based Buffer Overflow (CWE-122), Signed to Unsigned Conversion Error (CWE-195), Numeric Truncation Error (CWE-197), Buffer Over-read (CWE-126), and Unchecked Return Value to NULL Pointer Dereference (CWE-690). We use these code examples to train a vulnerability detection model and test it. We divide the dataset into a training set, a validation set, and a test set according to the ratio of 7:2:1.

4.2.3. Evaluation indicators

Vulnerability detection is a binary classification problem, so we use commonly used classification indicators Accuracy, TPR, FNR, and F1-Score. Accuracy represents the proportion of correctly predicted samples. TPR represents the proportion of the positive class in the predicted correct samples. F1-Score is the combined value of Precision and TPR. Their expressions are as follows.

² <https://pytorch.org/>.

³ <https://networkx.org/>.

⁴ <https://samate.nist.gov/SARD/>.

⁵ <https://nvd.nist.gov/>.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (10)$$

$$\text{TPR} = \frac{TP}{TP + FN} \quad (11)$$

$$\text{F1-Score} = \frac{2 * \text{Precision} * \text{TPR}}{\text{Precision} + \text{TPR}} \quad (12)$$

Where $\text{Precision} = TP / (TP + FP)$, TP is True Positive, TN is True Negative, FP is False Positive, FN is False Negative.

4.3. Experimental methods

For RQ1, we design experiment 1. Experiment 1: To verify the vulnerability detection performance of VulGAI, we compare with advanced deep learning-based vulnerability detection methods, which are the slice-level vulnerability detection method VulDePecker (Li et al., 2018), SySeVR (Li et al., 2021b) and the function-level vulnerability detection method Devign (Zhou et al., 2019) and VulCNN (Wu et al., 2022). Finally, we recorded the vulnerability detection results (Accuracy, TPR, and F1-Score values) of these methods.

For RQ2, we design experiments 2, 3, and 4. Experiment 2: To verify the performance impact of different code line thresholds on VulGAI, we performed a sensitivity analysis on the line threshold. In the image generation stage, we use 50, 80, 100, 120, 150, and 200 to generate images, respectively. That is, the image sizes are $3 \times 50 \times 128$, $3 \times 80 \times 128$, $3 \times 100 \times 128$, $3 \times 120 \times 128$, $3 \times 150 \times 128$, and $3 \times 200 \times 128$. Finally, we exploit the CNN model for vulnerability detection, and we record the F1-Score for these thresholds.

Experiment 3: To verify the impact of different layers of CPR2d (Conv2d + ZeroPad2d + ReLU) on the performance of VulGAI, we performed a sensitivity analysis on the number of layers of CPR2d. In the vulnerability detection model stage, we use 2, 3, 4, 6, and 8 layers to classify. Finally, we record the Accuracy of these tests.

Experiment 4: To verify the performance impact of different node embedding methods on VulGAI. We perform a sensitivity analysis on node embedding. In the node embedding stage, we choose Sent2vec (Pagliardini et al., 2018), Node2vec (Grover and Leskovec, 2016), and Struc2vec (Ribeiro et al., 2017) as node embedding methods. Finally, we utilize the CNN model for vulnerability detection. We recorded the Accuracy, TPR, and F1-Score values of these three methods.

For RQ3, we design experiment 5 and a case study. Experiment 5: To verify the vulnerability detection performance of VulGAI in the real world. We selected a real function example from the FFmpeg and QEMU projects, which we call ffmpeg (Zhou et al., 2019). The ffmpeg dataset contains 9776 vulnerable functions and 20494 neutral functions. We train VulGAI and VulCNN models on SARD_NVD. In the end, we got the best model to detect ffmpeg. We recorded the number of vulnerabilities detected by VulGAI and VulCNN and the detection time.

Case Study. We show some examples of vulnerabilities detected correctly. Finally, we explain a code example (image) using Gradient-weighted Class Activation Mapping++ (Chattopadhyay et al., 2018).

4.4. Experimental results

Results of Experiment 1. The performance of the six deep-learning vulnerability detection methods is shown in Fig. 6. It can be seen from the figure that VulGAI is the best in all three indicators. VulGAI achieved 83.3%, 84.3%, 15.7%, and 84.5% in Accuracy, TPR, FNR, and F1-Score, respectively. VulDePecker and SySeVR collect code fragments by slicing, and they use natural language processing methods for vulnerability detection. The performance of slice-level vulnerability detection methods is lower than other methods. We think that VulDePecker and SySeVR treat the slice as a piece of text, and the dependencies between code lines are not captured in the slice. The detection performance of VulCNN is similar to that of Devign. However, the complexity of the graph structure makes it difficult for Devign to expand to large-scale vulnerability detection. mVulPreter effectively fuses function-level

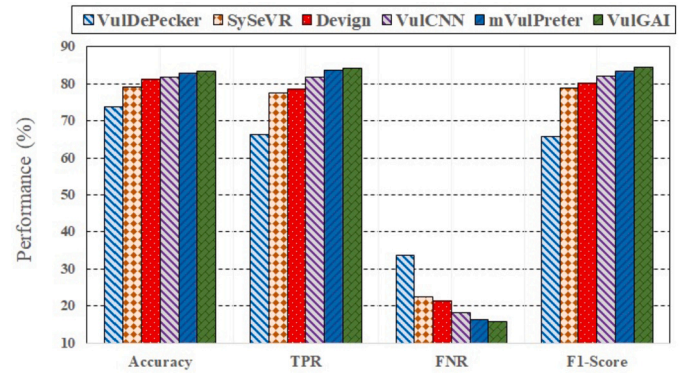


Fig. 6. Vulnerability detection performance of VulDePecker, SySeVR, Devign, VulCNN, mVulPreter, and VulGAI.

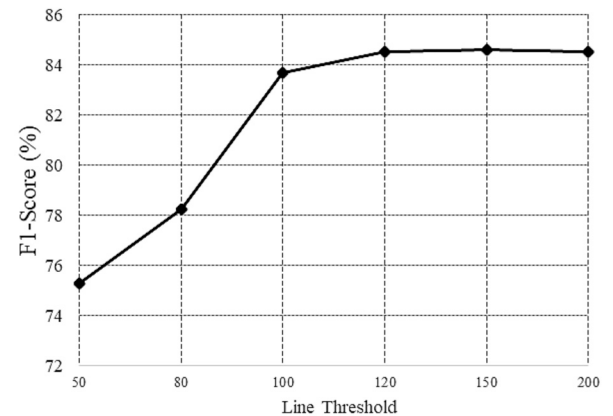


Fig. 7. F1-Score under different line thresholds.

and slice-level vulnerability detection models, which can capture more dependency information. Therefore, mVulPreter exhibits excellent performance. Within the margin of error, mVulPreter is close to VulGAI performance.

Specifically, VulGAI is about 2 percentage points higher than VulCNN. In terms of the computing performance of the model, VulGAI has 77178 computing parameters, however, VulCNN is 8.77 times more than VulGAI, and VulCNN has 676802 computing parameters. In terms of time performance, the average training time and testing time of one epoch of VulCNN are 182 s and 16 s, respectively. However, VulGAI only has 35 s and 4 s.

Results of Experiment 2. Fig. 7 shows F1-Score under different line thresholds (50, 80, 100, 120, 150, and 200). From Fig. 7, it can be found that at 120, F1-Score reached a steady state. The threshold is positively correlated with the F1-Score. The larger the threshold, the greater the F1-Score until it reaches the bottleneck. Therefore, this paper selects 120 as the line threshold to generate the image.

Results of Experiment 3. Fig. 9 shows the Accuracy under different CPR2d layers (2, 3, 4, 6, and 8). We found that at 4, Accuracy reaches the maximum. On layers 4–8, Accuracy fluctuates around 83%. The more layers, the more calculation parameters will be, and the training time will be longer. Considering the computing performance and time performance, this paper selects 4 layers of CPR2d.

Results of Experiment 4. Table 1 shows the vulnerability detection performance under the three embedding methods. As can be seen from the table, Sent2vec achieves optimal performance. Compared with Node2vec, the three indicators of Accuracy, TPR and F1-Score have increased by 9.5%, 9.4%, and 10.4%, respectively. We analyze the reasons for the performance gap from the perspective of class separation. We perform TSNE dimensionality reduction visualization on the feature vectors of the three embedding methods after the CNN

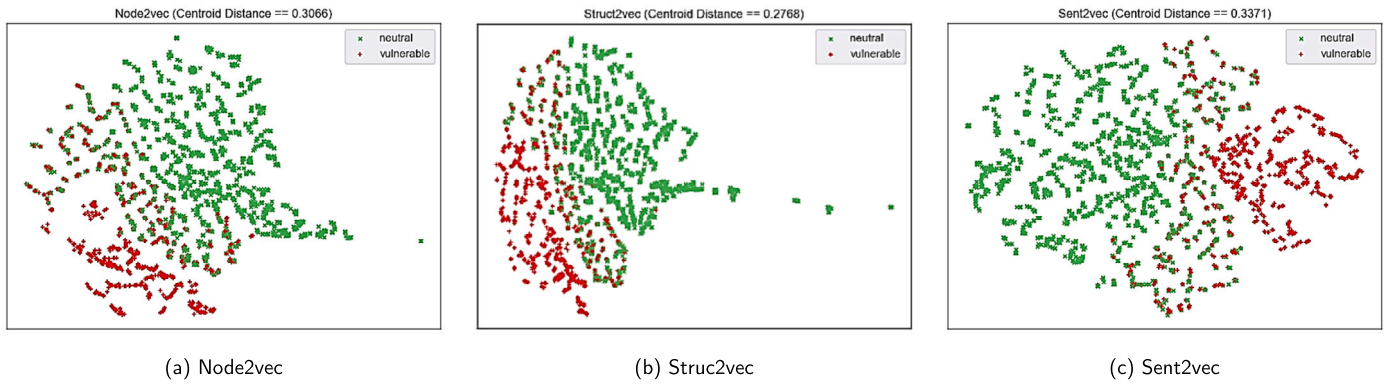


Fig. 8. TSNE visualization. Vulnerable and neutral sample feature distributions under Node2vec, Struc2vec, and Sent2vec.

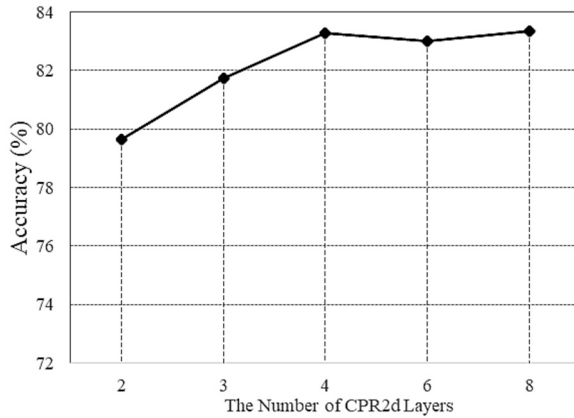


Fig. 9. Accuracy under different CPR2d layers.

Table 1
Vulnerability detection results in the real world.

Embedding Method	Accuracy	TPR	F1-Score
Sent2vec	83.3%	84.3%	84.5%
Node2vec	73.8%	73.9%	74.1%
Struc2vec	71.9%	71.8%	70.9%

Table 2
TP and Time of VulCNN and VulGAI for vulnerability detection.

Method	TP (True Positive)	Time (s)
VulGAI	3437	32 s
VulCNN	2324	125 s

model (Van der Maaten and Hinton, 2008). Fig. 8 shows the feature distributions of the vulnerable and neutral examples under Node2vec, Struc2vec, and Sent2vec. Where red “+” represents vulnerable examples and green “x” represents neutral examples. In addition, we calculated the centroid distance of the three methods. The larger the centroid distance, the better the separation between the two types (Mao et al., 2019). The centroid distances of Struc2vec, Node2vec, and Sent2vec are 0.2768, 0.3066, and 0.3371, respectively. The centroid distance of Sent2vec is the largest, which shows that its class separation is large. This explains the reason why Sent2vec performs well.

Results of Experiment 5. Table 2 presents the vulnerability detection results of VulGAI and VulCNN on the real-world dataset ffmqem. VulGAI detects 3437 vulnerabilities out of 9776 vulnerability examples, however, VulCNN only has 2324. The detection time of VulGAI is only 32 s, which is about 3.9 times faster than VulCNN.

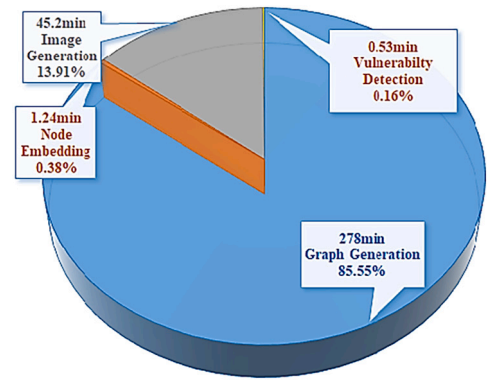


Fig. 10. The time of Graph Generation, Node Embedding, Image Generation, and Vulnerability Detection (minutes).

Case Study. This paper aims to achieve accuracy and efficiency in real-world vulnerability detection. We conducted a case study. We utilize the trained vulnerability detection model to detect the real-world ffmqem dataset. Specifically, the ffmqem dataset contains 9776 vulnerable functions and 20494 neutral functions. They have a total of 1875102 lines of code. Fig. 10 shows the timing of the four phases of VulGAI. We can generate 30,270 program dependency graphs within 278 minutes, and 30,270 images within 45.2 minutes. It can be seen from Fig. 10 that the graph generation phase and image generation phase account for 99.46% of the total time. However, we can complete the vulnerability detection within 0.53 min.

We manually analyzed the partial functions and compared them with real vulnerability patterns. If we find that both belong to the same pattern, we will judge it as a real vulnerability. Table 3 shows partial examples of vulnerabilities. We analyzed the types of these vulnerabilities and collected other important information (CVE-ID, BUGTRAQ ID, Vulnerability Type, Vulnerable File, and Release Date).

VulGAI is a vulnerability detection method based on the CNN model. Therefore, we can use CNN visualization techniques to explain our detection results. We use Grad-CAM++ to explain and visualize the Buffer Overflow vulnerability (image), as shown in Fig. 11. If “fgets” reads more than “SIZE” from “stdin”, the vulnerability will be triggered. It can be seen from Fig. 11 that the color of `if(fgets(inVuf, SIZE, stdin) != NULL)` is the darkest, which means that the code line may be a vulnerable statement. The result is consistent with our analysis.

5. Discussions

5.1. Threats to validity

Internal validity. It is related to the tuning of hyperparameters. Relying on the parameter experience in VulCNN, we empirically set

Table 3
Partial examples of vulnerabilities detected by VulGAI.

Target Project	CVE-ID	BUGTRAQ ID	Vulnerability Type	Vulnerable File	Release Date
FFmpeg	CVE-2017-14767	101019	Buffer Overflow	libavformat/rtpdec_h264.c	2017-09-27
FFmpeg	CVE-2015-1872	72644	DOS	libavcodec/utils.c	2015-02-18
FFmpeg	CVE-2016-10190	95986	Buffer Overflow	libavformat/http.c	2017-02-08
FFmpeg	CVE-2017-16840	101924	Out-Of-Bounds Read	libavcodec/libvpxenc.c	2017-11-20
FFmpeg	CVE-2017-14222	100701	DOS	libavformat/mov.c	2017-09-09
FFmpeg	CVE-2018-6621	102950	Out-Of-Bounds Read	libavcodec/utvideodec.c	2018-02-11
QEMU	CVE-2017-16845	101923	Out-of-bounds	hw/input/ps2.c	2017-11-20
QEMU	CVE-2016-10028	94981	Out-Of-Bounds Read	hw/display/virtio-gpu-3d.c	2017-02-22
QEMU	CVE-2017-5973	96220	DOS	hw/usb/hcd-xhci.c	2018-07-09
QEMU	CVE-2017-10806	99475	Buffer Overflow	hw/usb/redirect.c	2017-08-02
QEMU	CVE-2020-17380	970937	Buffer Overflow	hw/sd/sdhci.c	2021-01-30

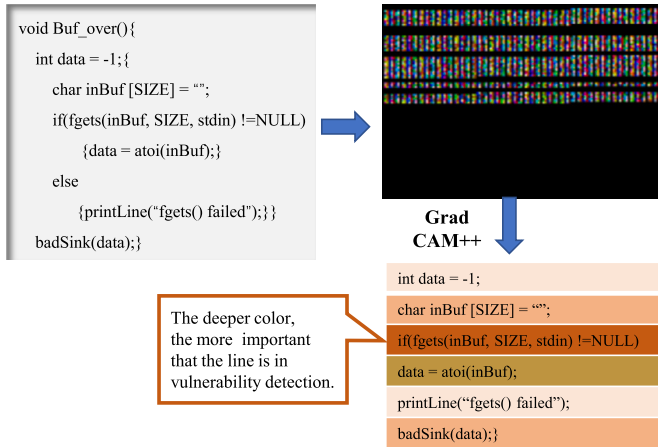


Fig. 11. Visualization of code explanation.

the relevant hyperparameters. To reduce the threat, we tested multiple experiments to set the line threshold and the number of layers of CPR2d.

External Threat. It is related to the quality of the dataset. SARD data is a synthetic function, and the synthetic function does not represent the entire program. We consider joining real-world NVD datasets to mitigate this threat.

Conclusion Threat. This paper makes improvements based on the problems existing in VulCNN. Therefore, VulCNN is our first comparison target. Under the same hardware and software environment, we adjust the parameters as much as possible to achieve optimal performance and ensure the fairness of the comparison.

Limitations. This paper is a function-level vulnerability detection method for C/C++ language. During code processing, we primarily consider lines of code. However, there is still a lack of finer-grained detection and interpretation during the detection process, such as the trigger path of the vulnerability. The three centralities of closeness centrality, degree centrality, and second_order centrality can distinguish the importance of nodes and retain program information. However, these three centralities are not directly related to image channels (RGB), it is just a simple correspondence method.

Future work. It is interesting to think about vulnerability detection from an image perspective. In the future, we expect to consider more fine-grained explanations from the vulnerability statement (Hin et al., 2022) and trigger path (Cheng et al., 2022). Perhaps, we can consider combining directional blurring techniques (Chen et al., 2018). For the conversion of centrality and channel, we need a more reasonable corresponding method, and we need to design them from the formula.

6. Related work

Graph-based code representation has been widely used in vulnerable code detection methods. We mainly introduce related work on

graph-based vulnerability detection. With the continuous development of deep learning, vulnerability detection technology has also become automated. VulDePecker (Li et al., 2018) and SySeVR (Li et al., 2021b) perform slice processing based on the program dependency graph to obtain code fragments. They utilize Bi-LSTM to obtain code fragment vectors and perform vulnerability detection. mVulPreter (Zou et al., 2022) also slices the program dependency graph, which utilizes GGNN to extract program semantics and detect vulnerabilities. FUNDED (Wang et al., 2020), Devign (Zhou et al., 2019), and Poem (Ye et al., 2020) are composite graph-based vulnerability detection methods, which consider multiple dependencies based on code property graphs. Finally, they utilize GGNN to extract graph global features and vulnerability detection. Reveal (Chakraborty et al., 2021) utilizes GGNN to extract feature vectors of code property graphs, which utilizes representation learning to improve vulnerability detection results. BGNN4VD (Cao et al., 2021) adds reverse edges to composite graphs to reverse aggregate node information, and it uses bidirectional gated graph neural networks for composite graph representation and vulnerability detection. DeepWukong (Cheng et al., 2021) processes the program dependency graph into multiple subgraphs according to interest points. Then, DeepWukong utilizes K-GNNS (Morris et al., 2019) for graph vector extraction. ACGVD (Li et al., 2021a) uses data control flow graphs, which are subgraphs of program dependency graphs. ACGVD designs node-level and path-level attention mechanisms to improve vulnerability detection performance. Linevd (Hin et al., 2022) utilizes data flow and control flow to reconstruct the code property graph, which removes some useless nodes. Finally, Linevd utilizes the graph attention network (hereinafter, GAT (Veličković et al., 2017)) for node classification. First, the above method processes the code into a graph structure. Next, they do some processing on the graph (splitting, reshaping, and adding edges). They perform code embeddings on graph nodes (such as, Sent2vec (Pagliardini et al., 2018), Node2vec (Grover and Leskovec, 2016), Word2vec (Mikolov et al., 2013), and Glove (Pennington et al., 2014)). Finally, they utilize graph neural networks (e.g., GGNN (Li et al., 2015), K-GNNS (Morris et al., 2019), and GAT (Veličković et al., 2017) to extract graph global feature vectors and detect vulnerabilities. However, VulCNN converts codes into images, which utilizes CNN models for vulnerability detection. According to the idea of VulCNN, this paper improves the node centrality and CNN model. We not only ensure the distinguishability of nodes but also improve computing performance and time performance.

7. Conclusion

Based on the inspiration of VulCNN, we propose a function-level vulnerability detection method. We analyze the lack of katz centrality, we use closeness centrality, degree centrality, and second_order centrality to generate images, which can distinguish node importance. We can preserve program details and visual explanations from the perspective of code lines. In addition, we design a more efficient CNN model, which reduces computational overhead and improves detection performance. We implement VulGAI and evaluated six methods (VulDePecker, SySeVR, Devign, VulCNN, mVulPreter, and VulGAI) on 40657 functions.

Experimental results show that VulGAI achieves higher Accuracy, TPR, and F1-Score than other advanced vulnerability detection techniques. Additionally, we conduct a comprehensive comparison of VulGAI and VulCNN on 30,270 real-world functions. VulGAI outperforms VulCNN by 1.48 times on TP. VulGAI has 8.77 times fewer calculation parameters than VulCNN. VulGAI is about 3.9 times faster than VulCNN in detection time. VulGAI can achieve the scalability and accuracy of code vulnerability scanning.

CRedit authorship contribution statement

Chunyang Zhang completed the experimental content and the manuscript of the paper, and Yang Xin provided the paper ideas, designed experiments and checked grammar.

Declaration of competing interest

Yang Xin reports financial support was provided by National Natural Science Foundation of China.

Data availability

The authors do not have permission to share data.

Acknowledgement

This paper was funded by the National Natural Science Foundation of China 62233003 and 2020YFB1708602.

References

- Cao, S., Sun, X., Bo, L., Wei, Y., Li, B., 2021. Bgnn4vd: constructing bidirectional graph neural-network for vulnerability detection. *Inf. Softw. Technol.* 136, 106576.
- Chakraborty, S., Krishna, R., Ding, Y., Ray, B., 2021. Deep learning based vulnerability detection: are we there yet. *IEEE Trans. Softw. Eng.*
- Chattopadhyay, A., Sarkar, A., Howlader, P., Balasubramanian, V.N., 2018. Grad-CAM++: generalized gradient-based visual explanations for deep convolutional networks. In: 2018 IEEE Winter Conference on Applications of Computer Vision. WACV. IEEE, pp. 839–847.
- Chen, H., Xue, Y., Li, Y., Chen, B., Xie, X., Wu, X., Liu, Y., 2018. Hawkeye: towards a desired directed grey-box fuzzer. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, pp. 2095–2108.
- Cheng, X., Wang, H., Hua, J., Xu, G., Sui, Y., 2021. DeepWukong: statically detecting software vulnerabilities using deep graph neural network. *ACM Trans. Softw. Eng. Methodol.* 30, 1–33.
- Cheng, X., Zhang, G., Wang, H., Sui, Y., 2022. Path-sensitive code embedding via contrastive learning for software vulnerability detection. In: Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 519–531.
- Fan, Y., Wan, C., Fu, C., Han, L., Xu, H., 2023. VDoTR: vulnerability detection based on tensor representation of comprehensive code graphs. *Comput. Secur.* 130, 103247.
- Freeman, L.C., et al., 2002. Centrality in social networks: conceptual clarification. In: *Social Network: Critical Concepts in Sociology*, vol. 1. Routledge, London, pp. 238–263.
- Ghaffarian, S.M., Shahriari, H.R., 2017. Software vulnerability analysis and discovery using machine-learning and data-mining techniques: a survey. *ACM Comput. Surv.* 50, 1–36.
- Grover, A., Leskovec, J., 2016. node2vec: scalable feature learning for networks. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 855–864.
- Hanif, H., Nasir, M.H.N.M., Ab Razak, M.F., Firdaus, A., Anuar, N.B., 2021. The rise of software vulnerability: taxonomy of software vulnerabilities detection and machine learning approaches. *J. Netw. Comput. Appl.* 179, 103009.
- Hin, D., Kan, A., Chen, H., Babar, M.A., 2022. LineVD: statement-level vulnerability detection using graph neural networks. In: Proceedings of the 19th International Conference on Mining Software Repositories, pp. 596–607.
- Katz, L., 1953. A new status index derived from sociometric analysis. *Psychometrika* 18, 39–43.
- Kermarrec, A.M., Le Merrer, E., Sericola, B., Trédan, G., 2011. Second order centrality: distributed assessment of nodes criticality in complex networks. *Comput. Commun.* 34, 619–628.
- Krizhevsky, A., Sutskever, I., Hinton, G.E., 2017. Imagenet classification with deep convolutional neural networks. *Commun. ACM* 60, 84–90.
- Le Merrer, E., Trédan, G., Le Scouarnec, N., 2014. Heuristical top-k: fast estimation of centralities in complex networks. *Inf. Process. Lett.* 114, 432–436.
- Li, M., Wu, Y., Zhang, B., Wen, Y., 2021a. ACGVD: vulnerability detection based on comprehensive graph via graph neural network with attention. In: *Information and Communications Security: Proceedings of the 23rd International Conference, Part i. ICICS 2021*, Chongqing, China, November 19–21, 2021, vol. 23. Springer, pp. 243–259.
- Li, Y., Tarlow, D., Brockschmidt, M., Zemel, R., 2015. Gated graph sequence neural networks. *arXiv preprint. arXiv:1511.05493*.
- Li, Z., Zou, D., Xu, S., Jin, H., Zhu, Y., Chen, Z., 2021b. SySeVR: a framework for using deep learning to detect software vulnerabilities. *IEEE Trans. Dependable Secure Comput.*
- Li, Z., Zou, D., Xu, S., Jin, H., Zhu, Y., Zhang, Y., Chen, Z., Li, D., 2021c. VulDeeLocator: a deep learning-based system for detecting and locating software vulnerabilities. *IEEE Trans. Dependable Secure Comput.*
- Li, Z., Zou, D., Xu, S., Ou, X., Jin, H., Wang, S., Deng, Z., Zhong, Y., 2018. VulDeePecker: a deep learning-based system for vulnerability detection. *arXiv preprint. arXiv:1801.01681*.
- Lin, G., Zhang, J., Luo, W., Pan, L., De Vel, O., Montague, P., Xiang, Y., 2019. Software vulnerability discovery via learning multi-domain knowledge bases. *IEEE Trans. Dependable Secure Comput.* 18, 2469–2485.
- Lin, G., Zhang, J., Luo, W., Pan, L., Xiang, Y., De Vel, O., Montague, P., 2018. Cross-project transfer representation learning for vulnerable function discovery. *IEEE Trans. Inform. Anal.* 14, 3289–3297.
- Van der Maaten, L., Hinton, G., 2008. Visualizing data using t-SNE. *J. Mach. Learn. Res.* 9.
- Mao, C., Zhong, Z., Yang, J., Vondrick, C., Ray, B., 2019. Metric learning for adversarial robustness. *Adv. Neural Inf. Process. Syst.* 32.
- Mikolov, T., Chen, K., Corrado, G., Dean, J., 2013. Efficient estimation of word representations in vector space. *arXiv preprint. arXiv:1301.3781*.
- Morris, C., Ritzert, M., Fey, M., Hamilton, W.L., Lenssen, J.E., Rattan, G., Grohe, M., 2019. Weisfeiler and Leman go neural: higher-order graph neural networks. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 4602–4609.
- Pagliardini, M., Gupta, P., Jaggi, M., 2018. Unsupervised learning of sentence embeddings using compositional n-gram features. In: *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*, pp. 528–540.
- Pennington, J., Socher, R., Manning, C.D., 2014. Glove: global vectors for word representation. In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing. EMNLP*, pp. 1532–1543.
- Perozzi, B., Al-Rfou, R., Skiena, S., 2014. Deepwalk: online learning of social representations. In: *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 701–710.
- Rastegari, M., Ordonez, V., Redmon, J., Farhadi, A., 2016. XNet: imagenet classification using binary convolutional neural networks. In: *Computer Vision—ECCV 2016: Proceedings of the 14th European Conference, Part IV. Amsterdam, the Netherlands, October 11–14, 2016. Springer*, pp. 525–542.
- Ribeiro, L.F., Saverese, P.H., Figueiredo, D.R., 2017. struc2vec: learning node representations from structural identity. In: *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 385–394.
- Selvaraju, R.R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., Batra, D., 2017. Grad-CAM: visual explanations from deep networks via gradient-based localization. In: *Proceedings of the IEEE International Conference on Computer Vision*, pp. 618–626.
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., Bengio, Y., 2017. Graph attention networks. *arXiv preprint. arXiv:1710.10903*.
- Wang, H., Ye, G., Tang, Z., Tan, S.H., Huang, S., Fang, D., Feng, Y., Bian, L., Wang, Z., 2020. Combining graph-based learning with automated data collection for code vulnerability detection. *IEEE Trans. Inf. Forensics Secur.* 16, 1943–1958.
- Watson, A., Ufuktepe, E., Palaniappan, K., 2022. Detecting software code vulnerabilities using 2d convolutional neural networks with program slicing feature maps. In: 2022 IEEE Applied Imagery Pattern Recognition Workshop. AIPR. IEEE, pp. 1–9.
- Wu, Y., Zou, D., Dou, S., Yang, W., Xu, D., Jin, H., 2022. VulCNN: an image-inspired scalable vulnerability detection system. In: *Proceedings of the 44th International Conference on Software Engineering*, pp. 2365–2376.
- Yamaguchi, F., Golde, N., Arp, D., Rieck, K., 2014. Modeling and discovering vulnerabilities with code property graphs. In: 2014 IEEE Symposium on Security and Privacy. IEEE, pp. 590–604.
- Ye, G., Tang, Z., Wang, H., Fang, D., Fang, J., Huang, S., Wang, Z., 2020. Deep program structure modeling through multi-relational graph-based learning. In: *Proceedings of the ACM International Conference on Parallel Architectures and Compilation Techniques*, pp. 111–123.
- Zhou, Y., Liu, S., Siow, J., Du, X., Liu, Y., 2019. Devign: effective vulnerability identification by learning comprehensive program semantics via graph neural networks. *Adv. Neural Inf. Process. Syst.* 32.
- Zou, D., Hu, Y., Li, W., Wu, Y., Zhao, H., Jin, H., 2022. mVulPreter: a multi-granularity vulnerability detection system with interpretations. *IEEE Trans. Dependable Secure Comput.*
- Zou, D., Wang, S., Xu, S., Li, Z., Jin, H., 2019. μ VulDeePecker: a deep learning-based system for multiclass vulnerability detection. *IEEE Trans. Dependable Secure Comput.* 18, 2224–2236.